



RAPPORT DE STAGE

Filière : Big Data, Data science & AI

PROJET DE FIN D'ANNÉE SmartTimetable AI

Système Intelligent de Génération et d'Optimisation des
Emplois du Temps Académiques

Encadré par :
RASSAM Latifa

Réalisé par :
Amakassou Aziza

Soutenue Le : 07/09/2026

Année Universitaire 2026- 2027

DEDICACES

Je dédie ce modeste travail :

À mes chers parents,

Pour leur amour, leur soutien indéfectible et leurs sacrifices qui ont rendu possible la réalisation de ce projet.

À tous mes enseignants et encadrants,

Qui m'ont guidé tout au long de ce parcours.

À tous ceux qui, de près ou de loin,

Ont contribué à l'aboutissement de ce travail.

REMERCIEMENTS

Avant d'entamer la rédaction de ce rapport, je tiens à exprimer ma profonde gratitude à travers ces lignes.

Je remercie chaleureusement **l'École Nationale Supérieure de l'Intelligence Artificielle et des Sciences des Données Taroudant (ENSIASDT)** pour la qualité de l'enseignement qu'elle dispense, ainsi que pour son engagement à nous préparer aux exigences du monde professionnel à travers des expériences concrètes comme les stages. Cette opportunité m'a permis d'appliquer mes acquis dans un environnement réel et enrichissant.

Je souhaite également adresser mes plus vifs remerciements à **Madame Latifa RASSAM**, mon encadrante lors du déroulement du stage, pour son encadrement bienveillant, ses conseils pertinents et l'intérêt qu'il a porté à mon travail. Ses remarques m'ont beaucoup aidée à approfondir mes réflexions et à mener à bien ce projet.

Résumé

La planification des emplois du temps académiques constitue un problème combinatoire complexe, particulièrement sensible dans les établissements d'enseignement supérieur où se croisent de multiples contraintes liées aux enseignants, aux salles, aux groupes d'étudiants et aux créneaux horaires disponibles.

Ce Projet de Fin d'Année présente la conception et le développement de SmartTimetable AI, une plateforme web intelligente destinée à automatiser la génération et l'optimisation de ces emplois du temps à l'aide de techniques d'intelligence artificielle issues du domaine de l'optimisation combinatoire.

Le système repose sur une architecture en couches associant une API REST développée avec FastAPI, une base de données relationnelle PostgreSQL modélisée pour représenter fidèlement les entités académiques (départements, filières, groupes, enseignants, salles, modules, créneaux) et leurs contraintes métier, ainsi qu'une interface web permettant la gestion complète des données et le pilotage de la génération. Le cœur du système repose sur un moteur d'optimisation découpé, implémentant et comparant trois approches algorithmiques : une méthode exacte de programmation par contraintes (**CP-SAT**, **Google OR-Tools**) et deux métaheuristiques de recherche locale, **la recherche tabou** et **le recuit simulé**. Ces trois algorithmes sont évalués selon trois critères homogènes : le temps de calcul, le respect strict des contraintes dures et le score de pénalité associé aux contraintes souples.

Les résultats obtenus démontrent la faisabilité d'une génération automatique d'emplois du temps respectant l'ensemble des contraintes dures définies (non-conflit des enseignants, des salles et des groupes, adéquation des capacités et des types de salles, couverture du volume horaire), tout en offrant une base comparative rigoureuse entre méthode exacte et métaheuristiques, directement exploitable dans le cadre d'une étude académique sur les méthodes d'intelligence artificielle appliquées à l'optimisation combinatoire.

Abstract

Academic timetabling is a complex combinatorial problem, particularly challenging in higher education institutions where numerous constraints related to teachers, rooms, student groups and available time slots must be simultaneously satisfied. This End-of-Year Project presents the design and development of SmartTimetable AI, an intelligent web platform aimed at automating the generation and optimization of academic timetables using artificial intelligence techniques drawn from combinatorial optimization.

The system relies on a layered architecture combining a REST API built with FastAPI, a relational PostgreSQL database modeling the core academic entities (departments, programs, groups, teachers, rooms, courses, time slots) and their business constraints, and a web interface built with React and TypeScript enabling full data management and generation control. The core of the system is a decoupled optimization engine implementing and comparing three algorithmic approaches: an exact constraint-programming method (CP-SAT, Google OR-Tools) and two local-search metaheuristics, tabu search and simulated annealing. All three algorithms are evaluated on three homogeneous criteria: computation time, strict compliance with hard constraints, and the penalty score associated with soft constraints.

The results demonstrate the feasibility of automatically generating timetables that comply with all defined hard constraints (no teacher, room or group conflicts, adequate room capacity and type, coverage of the required teaching hours), while providing a rigorous comparative basis between the exact method and the metaheuristics, directly usable as an academic study of artificial intelligence methods applied to combinatorial optimization.

Table des matières

I.	Chapitre 1 : Étude de l'existant et analyse des besoins	14
A.	Contexte du projet	14
B.	Problématique.....	14
C.	Étude de l'existant.....	14
1.	Solutions existantes	14
2.	Analyse comparative	15
3.	Limites.....	15
D.	Besoins fonctionnels	15
E.	Besoins non fonctionnels	16
F.	Acteurs	16
G.	Cas d'utilisation	18
1.	Diagramme globale	18
2.	Description des principaux cas d'utilisation	18
H.	Conclusion.....	19
II.	Chapitre 2 : Conception du système.....	20
A.	Architecture générale	20
B.	Choix technologiques.....	20
C.	Diagramme de classes	21
D.	Diagrammes de séquence	22
1.	Séquence : génération d'un emploi du temps	22
2.	Séquence : création d'une séance avec validation des contraintes	23
E.	Diagramme de composants	24
F.	Diagramme de déploiement	25
G.	Modèle de base de données	26
1.	Contraintes d'unicité.....	26
2.	Les contraintes dures (obligatoires)	27
3.	Contraintes souples :	28
H.	Conclusion.....	28
III.	Chapitre 3 – Réalisation	29
A.	Environnement de développement	29
B.	Architecture logicielle	29
C.	Implémentation des fonctionnalités	30
1.	Gestion des entités de référence	30

2.	Gestion des créneaux et des indisponibilités	31
3.	Gestion des séances et validation des contraintes dures	31
4.	Import de données en masse.....	32
5.	Génération des séances.....	32
D.	Implémentation des fonctionnalités d'intelligence artificielle.....	34
1.	Programmation par contraintes, CP-SAT (Google OR-Tools).....	34
2.	Recherche tabou	35
3.	Recuit simulé.....	36
4.	Fonction d'évaluation partagée.....	37
5.	Étude comparative.....	38
6.	Précision sur le périmètre des fonctionnalités d'intelligence artificielle	38
7.	Sécurité.....	39
E.	Conclusion.....	39
IV.	Chapitre 4 : Tests et évaluation	40
A.	Stratégie de test	40
B.	Jeux de tests.....	40
C.	Résultats	41
D.	Difficultés rencontrées	41
1.	Dérive de schéma sans outil de migration.....	42
2.	Verrouillage de transaction en base de données	42
3.	Infaisabilité mathématique d'un jeu de données de test	42
E.	Perspectives d'amélioration	42
V.	Conclusion générale	44
A.	Bilan	44
B.	Apports.....	44
C.	Limites.....	44
D.	Perspectives	45
E.	Bibliographie	45

Liste des figures

Figure 1: Digramme de cas d'utilisation.....	18
Figure 2 : Diagramme de classe	22
Figure 3 : Diagrammes de séquence du scénario génération d'un emploi du temps	23
Figure 4 : génération d'un emploi du temps du scénario création d'une séance avec validation des contraintes.....	24
Figure 5 : Diagramme de composants	25
Figure 6 : Diagramme de déploiement	26
Figure 7: L'arborescence du backend	29
Figure 8 : L'arborescence du frontend.....	30
Figure 9 :Onglet Configuration et Gestion.....	31
Figure 10 : Onglet Chargement de données	32
Figure 11: Génère, compare et publie l'emploi du temps à partir des données saisies.	33
Figure 12 : Planning généré.....	38
Figure 13 : tableau comparatif des trois algorithmes	38

Liste des tableaux

Tableau 1: Analyse comparative des solutions existantes	15
Tableau 2 : Acteurs du système	17
Tableau 3: Choix technologiques et justifications.....	20
Tableau 4: Contraintes d'unicité	27
Tableau 5 : Les contraintes dures (obligatoires).....	27
Tableau 6 : Contraintes souples	28
Tableau 7: Organisation du backend	29

INTRODUCTION GENERALE

Contexte

La construction des emplois du temps est une activité récurrente et critique dans tout établissement d'enseignement supérieur. Elle intervient à chaque début de semestre et doit concilier un grand nombre de paramètres : la disponibilité des enseignants, la capacité et la vocation pédagogique des salles (amphithéâtre, salle de cours, salle de travaux pratiques), les effectifs des groupes et sous-groupes d'étudiants, le volume horaire imposé par chaque module, ainsi que des préférences pédagogiques visant à améliorer la qualité de vie académique (éviter les créneaux tardifs, préserver une pause déjeuner, limiter les temps morts entre deux séances).

Réalisée manuellement, cette tâche est chronophage et sujette à erreur : un simple oubli de contrainte peut engendrer un conflit d'occupation de salle ou une double affectation d'un enseignant. À mesure que le nombre de filières, de groupes et de modules augmente, la complexité combinatoire du problème croît de manière exponentielle, rendant une résolution purement manuelle de moins en moins réaliste. C'est dans ce contexte que s'inscrit le projet SmartTimetable AI : proposer une plateforme capable d'automatiser cette planification à l'aide de techniques d'intelligence artificielle et d'optimisation combinatoire, tout en conservant à l'utilisateur humain la maîtrise des données et des choix stratégiques.

Problématique

Le problème de construction d'un emploi du temps académique appartient à la famille des problèmes de satisfaction de contraintes (CSP – Constraint Satisfaction Problem), reconnus comme NP-difficiles dans le cas général. Il s'agit d'affecter à chaque séance d'enseignement un créneau horaire et une salle, de telle sorte que l'ensemble des contraintes obligatoires (dites contraintes dures) soit strictement respecté, tout en minimisant les violations des contraintes de confort (dites contraintes souples). La problématique centrale de ce projet peut donc être formulée ainsi : comment concevoir une plateforme logicielle capable de modéliser fidèlement les données académiques et leurs contraintes, puis de générer automatiquement, en un temps raisonnable, un emploi du temps valide et de bonne qualité, tout en permettant une comparaison objective entre différentes approches algorithmiques d'intelligence artificielle ?

Objectifs fonctionnels et techniques

Ce projet poursuit plusieurs objectifs complémentaires :

- Concevoir un modèle de données relationnel capable de représenter fidèlement l'ensemble des entités académiques (départements, filières, groupes, enseignants, salles, modules, créneaux) ainsi que leurs relations, y compris les cas de mutualisation de modules entre plusieurs filières.
- Développer une API REST robuste, appliquant les contraintes dures directement au niveau de la base de données et de la couche métier, avec une gestion d'erreurs explicite et exploitable par une interface utilisateur.
- Concevoir et implémenter un moteur d'optimisation découplé de l'infrastructure web, capable d'être testé et validé indépendamment, et respectant une interface commune permettant de substituer un algorithme à un autre sans impact sur le reste du système.
- Implémenter et comparer trois algorithmes d'intelligence artificielle représentatifs de deux familles distinctes de résolution : une méthode exacte de programmation par contraintes (CP-SAT) et deux métaheuristiques de recherche locale (recherche tabou et recuit simulé).

- Développer une interface web complète permettant la saisie, l'import en masse, la visualisation et l'impression des emplois du temps générés, aussi bien par filière que par enseignant.

Méthodologie

Le développement du projet a suivi une démarche itérative et incrémentale, structurée en étapes progressives validées les unes après les autres : modélisation des données de référence, mise en place des opérations de gestion (CRUD) pour chaque entité, implémentation des contraintes métier au niveau du service et de la base de données, conception et développement du moteur d'optimisation en isolation avant son intégration à l'API, puis développement de l'interface utilisateur. Cette approche a permis de valider chaque brique fonctionnelle indépendamment avant de l'intégrer à l'ensemble du système, réduisant ainsi les risques associés à la complexité croissante du projet.

Organisation du rapport

Ce rapport est structuré en quatre chapitres. Le premier chapitre présente une étude de l'existant et une analyse détaillée des besoins fonctionnels et non fonctionnels du système. Le deuxième chapitre décrit la conception du système : architecture générale, choix technologiques justifiés, modélisation des données et diagrammes de conception. Le troisième chapitre détaille la réalisation du projet, en particulier l'implémentation des fonctionnalités métier et des algorithmes d'intelligence artificielle constituant le cœur du système. Le quatrième chapitre présente la stratégie de test adoptée, les résultats obtenus, les difficultés rencontrées au cours du développement et les perspectives d'amélioration envisagées.

I. Chapitre 1 : Étude de l'existant et analyse des besoins

A. Contexte du projet

Le présent projet s'inscrit dans le cadre du Projet de Fin d'Année (PFA) de la filière Sciences des Données, Big Data & Intelligence Artificielle. Il répond à un besoin concret et récurrent au sein des établissements d'enseignement supérieur : la génération d'un emploi du temps académique cohérent, respectant les contraintes organisationnelles de l'établissement (disponibilité des enseignants et des salles) tout en optimisant la qualité pédagogique du planning obtenu (répartition équilibrée des séances, limitation des créneaux tardifs, préservation d'une pause déjeuner).

B. Problématique

La construction manuelle d'un emploi du temps mobilise un temps administratif important et reste sujette à erreurs humaines, en particulier lorsque le nombre de filières, de groupes, de modules et d'intervenants augmente. La problématique traitée dans ce chapitre consiste à identifier précisément les besoins fonctionnels et non fonctionnels que doit satisfaire une plateforme visant à automatiser cette tâche, avant d'aborder, dans les chapitres suivants, les choix de conception et d'implémentation qui en découlent.

C. Étude de l'existant

1. Solutions existantes

Plusieurs catégories de solutions coexistent actuellement pour la gestion des emplois du temps académiques. La première, la plus répandue dans les petits établissements, repose sur des outils bureautiques généralistes tels que les tableurs (Microsoft Excel, Google Sheets), utilisés pour construire manuellement le planning. La deuxième catégorie regroupe les progiciels spécialisés commerciaux, tels que Hyperplanning ou ADE Scheduling, qui proposent une gestion assistée mais reposent le plus souvent sur des algorithmes propriétaires peu documentés et nécessitent des licences coûteuses. Une troisième catégorie regroupe des outils open source orientés recherche opérationnelle, tels que FET (Free Timetabling Software), qui implémentent des heuristiques de résolution de contraintes mais offrent une intégration limitée avec les systèmes d'information existants d'un établissement et une personnalisation restreinte des contraintes métier.

2. Analyse comparative

Tableau 1: Analyse comparative des solutions existantes

Critère	Tableur manuel	Progiciel commercial	Outil open source	Smart TimeTable
Automatisation de la génération	Non	Partielle	Oui	Oui
Comparaison de plusieurs algorithmes	Non applicable	Non	Non	Oui
Personnalisation des contraintes métier	Totale(manuelle)	Limitée	Limitée	Total (code source)
Interface web moderne	Non	Variable	Rarement	Oui
Coût de licence	Faible	Élevé	Nul	Null (develeppent propre)
Import en masse(Excel)	Non applicable	Variable	Variable	Oui

3. Limites

Cette étude comparative met en évidence deux limites majeures des solutions existantes vis-à-vis des objectifs pédagogiques et techniques du présent projet. D'une part, aucune des solutions étudiées ne propose nativement une comparaison objective entre plusieurs approches algorithmiques d'intelligence artificielle appliquées au même jeu de données, ce qui constitue pourtant un axe d'étude central de ce PFA. D'autre part, les solutions propriétaires n'offrent pas de visibilité sur la logique de résolution employée, ce qui limite leur valeur pédagogique et leur adaptabilité aux contraintes spécifiques d'un établissement donné.

D. Besoins fonctionnels

L'analyse des besoins a permis d'identifier les fonctionnalités suivantes, directement issues du cahier des charges du projet :

- Gérer les entités académiques de référence : départements, filières, groupes, enseignants, salles et modules, y compris la mutualisation de modules entre plusieurs filières.
- Gérer les créneaux horaires disponibles pour la planification hebdomadaire.
- Déclarer les indisponibilités des enseignants sur des créneaux donnés.

- Définir et paramétrer les contraintes dures et souples du problème d'optimisation.
- Lancer la génération automatique d'un emploi du temps selon l'algorithme choisi parmi les méthodes disponibles.
- Comparer les performances de plusieurs algorithmes sur un même jeu de données.
- Visualiser l'emploi du temps généré sous forme de grille, filtrée par filière ou par enseignant.
- Imprimer ou exporter l'emploi du temps généré.
- Importer en masse les données de référence à partir de fichiers Excel.
- Modifier manuellement une séance générée en cas de besoin ponctuel.

E. Besoins non fonctionnels

1. **Performance** : la génération par méthode exacte doit produire un résultat dans un délai borné et maîtrisé, y compris en cas d'absence de solution valide.
2. **Fiabilité** : les contraintes dures relatives aux conflits d'enseignants, de salles et de groupes doivent être garanties au niveau de la base de données, indépendamment de la couche applicative.
3. **Clarté des erreurs** : toute violation de contrainte, tout conflit ou toute donnée manquante doit être signalée à l'utilisateur par un message explicite, exploitable directement depuis l'interface
4. **Modularité** : le moteur d'optimisation doit être découplé du reste de l'application, testable de façon autonome et interchangeable entre plusieurs algorithmes sans modification du reste du système.
5. **Ergonomie** : l'interface doit permettre une saisie guidée des données dans l'ordre logique de leurs dépendances, afin d'éviter les erreurs de configuration.
6. **Portabilité** : le système doit pouvoir être déployé sur un poste de développement standard, sans dépendance à un environnement propriétaire.

F. Acteurs

Le système identifie quatre acteurs principaux, conformément au diagramme de cas d'utilisation :

Tableau 2 : Acteurs du système

Acteur	Role principal
Administrateur	Gère l'ensemble des données de référence, définit les contraintes, déclenche la génération et valide le planning final
Enseignant	Consulte son planning personnel et déclare ses indisponibilités
Étudiant	Consulte le planning de son groupe et reçoit les notifications associées.
System AI (moteur d'optimisation)	Acteur logiciel qui valide les contraintes et optimise l'emploi du temps lors de la génération.

G. Cas d'utilisation

1. Diagramme globale

[1]

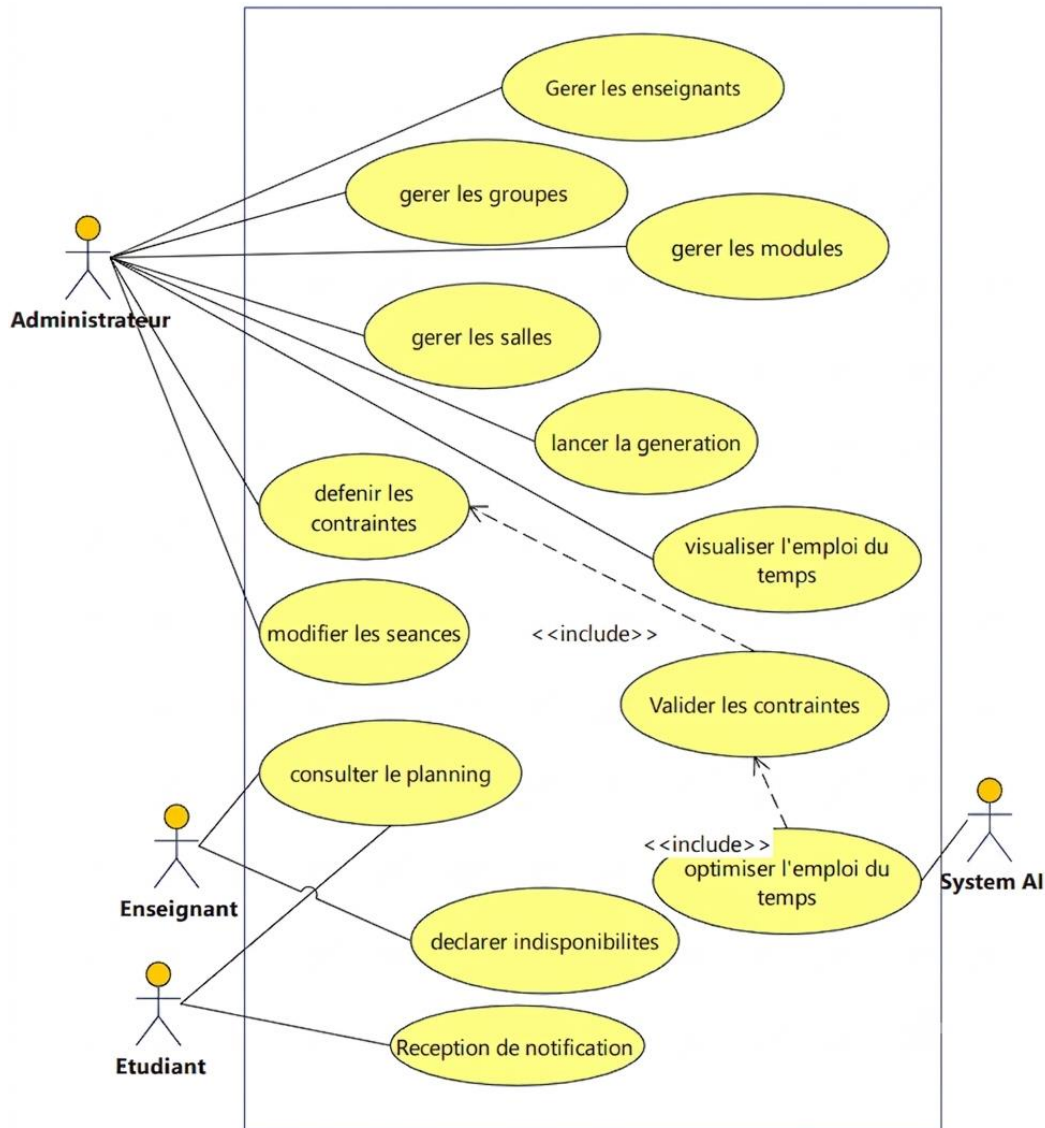


Figure 1: Digramme de cas d'utilisation

2. Description des principaux cas d'utilisation

- Gérer les enseignants / groupes / modules / salles : opérations de création, consultation, modification et suppression (CRUD) pour chaque entité de référence.
- Définir les contraintes : paramétrage des contraintes dures et souples exploitées par le moteur d'optimisation.

- Lancer la génération : déclenchement du calcul d'un emploi du temps par l'algorithme sélectionné.
- Valider les contraintes (inclut Optimiser l'emploi du temps) : le System AI vérifié le respect des contraintes dures avant de retourner la solution optimisée.
- Consultation du planning généré sous forme de grille, par filière ou par enseignant.
- Déclarer indisponibilités : un enseignant signale les créneaux sur lesquels il n'est pas disponible.
- Modifier les séances : ajustement manuel d'une séance après génération.

H. Conclusion

Cette étude de l'existant et cette analyse des besoins ont permis de dégager les exigences fonctionnelles et non fonctionnelles structurant l'ensemble des choix de conception présentés dans le chapitre suivant. Elles mettent notamment en évidence la nécessité d'une architecture modulaire, capable d'accueillir plusieurs algorithmes d'optimisation interchangeables, condition indispensable à l'étude comparative constituant le cœur scientifique de ce projet.

II. Chapitre 2 : Conception du système

A. Architecture générale

L'architecture retenue repose sur une séparation stricte entre trois couches logicielles indépendantes, communiquant exclusivement via des interfaces bien définies : une couche de présentation (le client web), une couche applicative exposant une API REST, et une couche de persistance relationnelle. À cette architecture classique s'ajoute une quatrième couche, spécifique à ce projet, dédiée au moteur d'optimisation : celui-ci est volontairement isolé de toute dépendance vis-à-vis du framework web ou de l'ORM, afin de pouvoir être développé, testé et exécuté de manière totalement autonome, y compris en ligne de commande, indépendamment du serveur applicatif.

Le client web (React/TypeScript) communique avec l'API REST (FastAPI) exclusivement via des requêtes HTTP au format JSON. L'API REST orchestre les échanges avec la base de données relationnelle (PostgreSQL) au travers de l'ORM SQLAlchemy, et délègue au moteur d'optimisation la résolution du problème de planification lorsqu'une génération est demandée. Le moteur d'optimisation reçoit en entrée une structure de données neutre (indépendante du schéma de base de données) et retourne une solution sous une forme tout aussi neutre, charge ensuite à la couche applicative de la persister.

B. Choix technologiques

Tableau 3: Choix technologiques et justifications

Composant	Technologie retenue	Justification synthétique
Langage du moteur d'optimisation	Python	Framework typé, asynchrone, générant automatiquement une documentation interactive (OpenAPI/Swagger) facilitant les tests durant tout le développement.
API REST	FastAPI [2]	Framework typé, asynchrone, générant automatiquement une documentation interactive (OpenAPI/Swagger) facilitant les tests durant tout le développement.
ORM	SQLAlchemy	ORM Python de référence, offrant un contrôle fin des contraintes relationnelles (clés étrangères,

		contraintes d'unicité) directement traduites en contraintes SQL natives.
Base de données	PostgreSQL	Système de gestion de base de données relationnel robuste, supportant nativement l'intégrité référentielle et les contraintes d'unicité composites utilisées pour garantir certaines contraintes dures dès la couche de persistance
Moteur d'optimisation exact	Google OR-Tools(CP-SAT) [3]	Solveur de programmation par contraintes open source, reconnu pour ses performances sur les problèmes combinatoires de type emploi du temps.
Métaheuristiques	Implémentation Python native (recherche tabou, recuit simulé)	Permet un contrôle total de la fonction de coût, de la structure de voisinage et des paramètres, indispensable à une comparaison rigoureuse avec la méthode exacte."],
Interface web	React 18 avec TypeScript	Typage statique réduisant les erreurs d'intégration avec l'API ; écosystème mature pour la construction d'interfaces de gestion de données complexes.
Outillage de build frontend	Vite	Démarrage et rechargement à chaud quasi instantanés, adaptés à un développement itératif rapide
Mise en forme	Tailwind CSS	Approche utilitaire permettant une cohérence visuelle stricte sans surcharge de feuilles de style dédiées

C. Diagramme de classes

Le modèle de classes du système s'articule autour de dix entités principales ; L'entité **Departement** constitue la racine hiérarchique : elle regroupe des **Filiere**, elle-même reliée aux **Groupe** qui la suivent. Les **Module** sont reliés aux **Filiere** par une relation plusieurs-à-plusieurs, permettant de représenter la mutualisation d'un même module entre plusieurs filières (tronc commun). L'entité **Seance** constitue la table pivot du système : elle relie un **Module**, un **Enseignant**, une **Salle**, un **Groupe** et un **Creneau**, et porte les contraintes d'unicité garantissant

l'absence de conflit. L'entité **Indisponibilite** relie un Enseignant à un Creneau sur lequel il n'est pas disponible.

Enfin, l'entité **Contrainte** constitue une table de paramétrage exploitée par le moteur d'optimisation pour pondérer les contraintes souples.

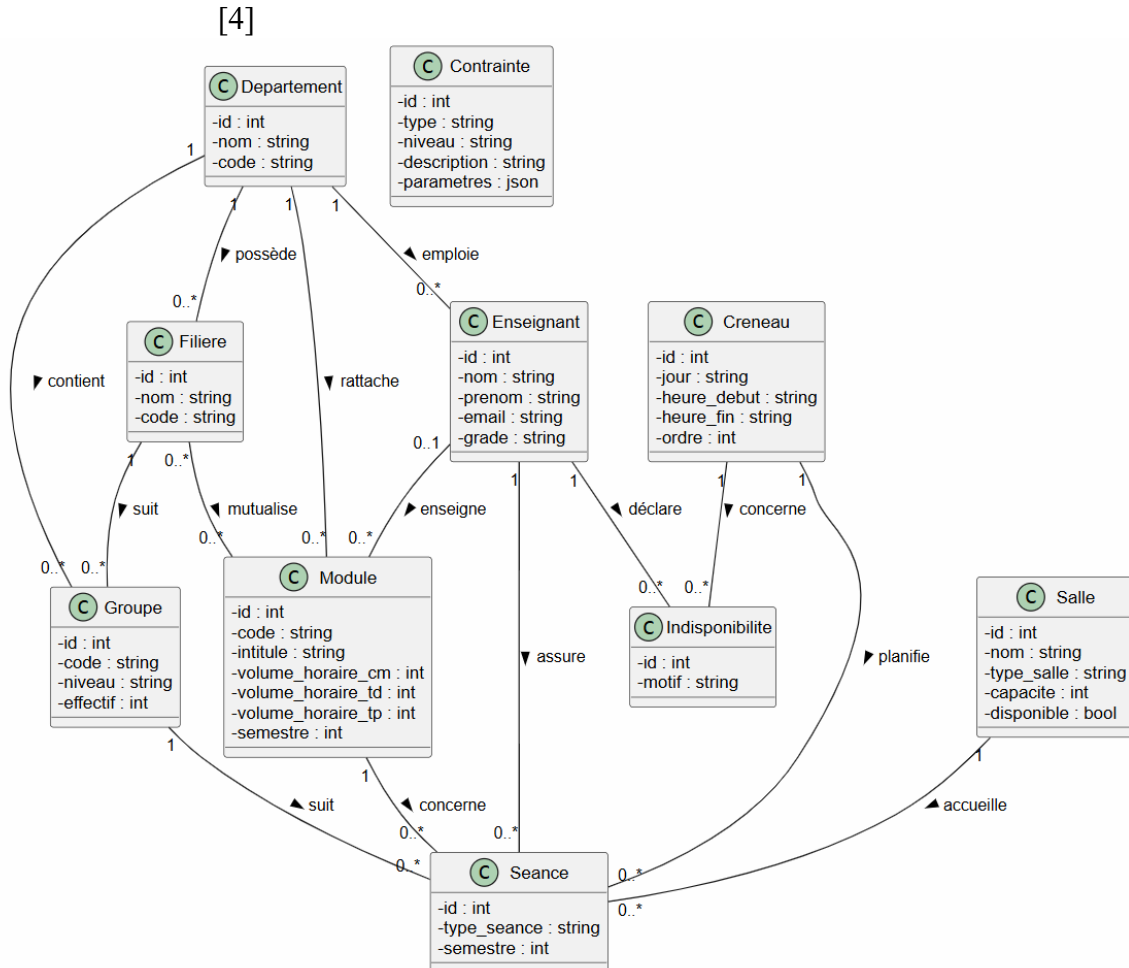


Figure 2 : Diagramme de classe

D. Diagrammes de séquence

1. Séquence : génération d'un emploi du temps

➔ Lorsque l'administrateur déclenche une génération, l'interface envoie une requête à l'endpoint de génération en précisant l'algorithme choisi. La couche applicative charge l'ensemble des données de référence depuis la base (modules, groupes, salles, créneaux, indisponibilités), les convertit vers la structure neutre attendue par le moteur d'optimisation, puis invoque l'algorithme sélectionné. Le moteur retourne une solution accompagnée d'indicateurs de performance (temps de calcul, respect des contraintes dures, score de pénalité). La couche applicative supprime le

planning précédent, persiste les nouvelles séances, puis retourne le résultat à l'interface, qui l'affiche à l'utilisateur.

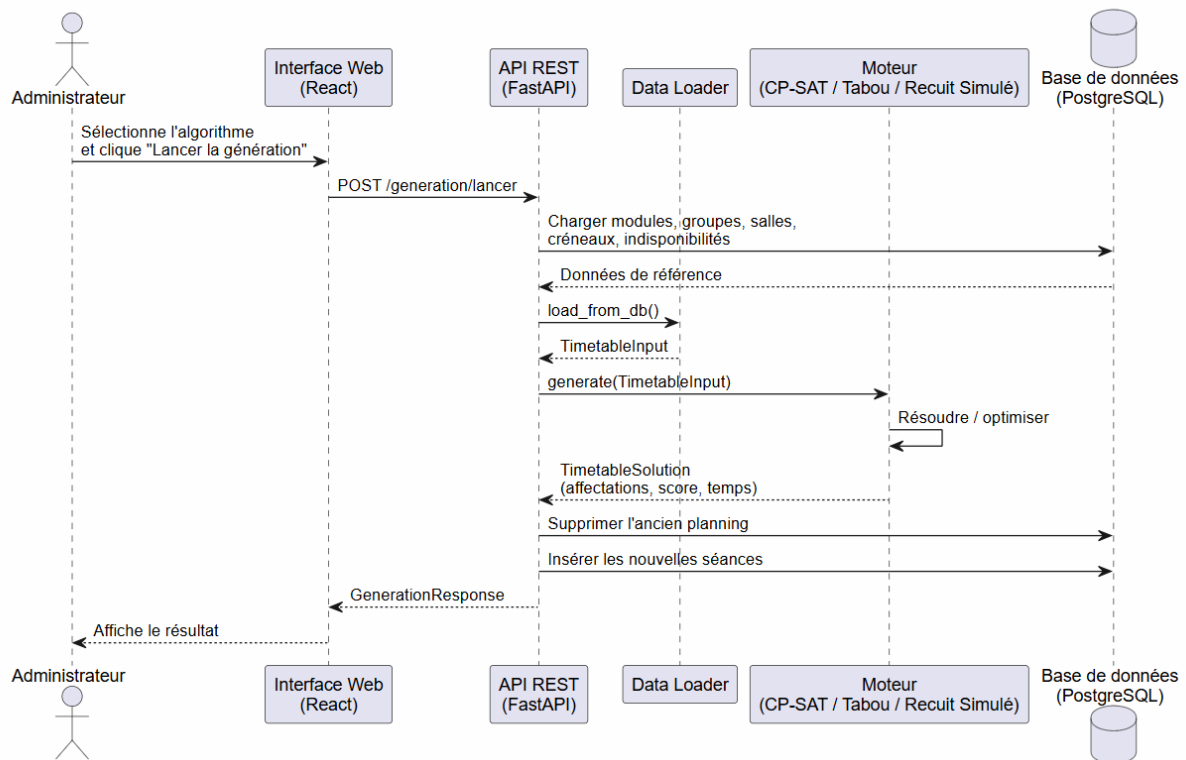


Figure 3 : Diagrammes de séquence du scénario génération d'un emploi du temps

2. Séquence : création d'une séance avec validation des contraintes

- ➔ Lors de la création manuelle d'une séance, la couche applicative vérifie d'abord l'existence des références (module, enseignant, salle, groupe, créneau). Elle invoque ensuite la couche de validation métier, qui contrôle l'adéquation entre l'effectif du groupe et la capacité de la salle, ainsi que l'adéquation entre le type de séance et le type de salle. La persistance en base déclenche enfin la vérification des contraintes d'unicité (non-conflit enseignant, salle, groupe), gérée nativement par le système de gestion de base de données.

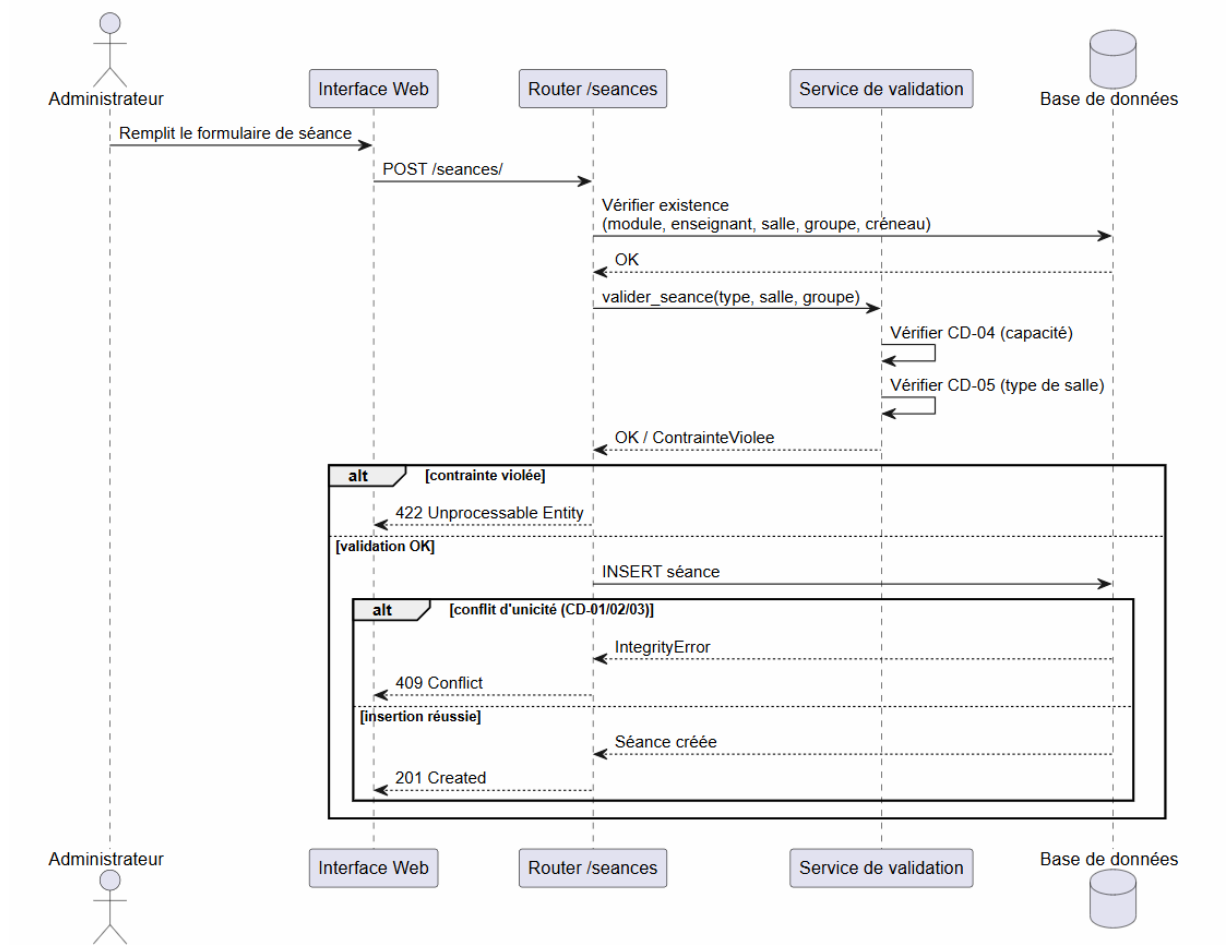


Figure 4 : génération d'un emploi du temps du scénario création d'une séance avec validation des contraintes

E. Diagramme de composants

Le système est décomposé en composants logiciels faiblement couplés : un composant d'**interface web**, un composant d'**API** regroupant les **routeurs REST** organisés par entité métier, un composant de services regroupant la **logique de validation métier**, un composant de persistance regroupant les **modèles ORM**, et un composant d'**optimisation**, lui-même subdivisé en un sous-composant de **chargement des données**, un sous-composant d'**évaluation des contraintes**, et un sous-composant regroupant les trois moteurs algorithmiques

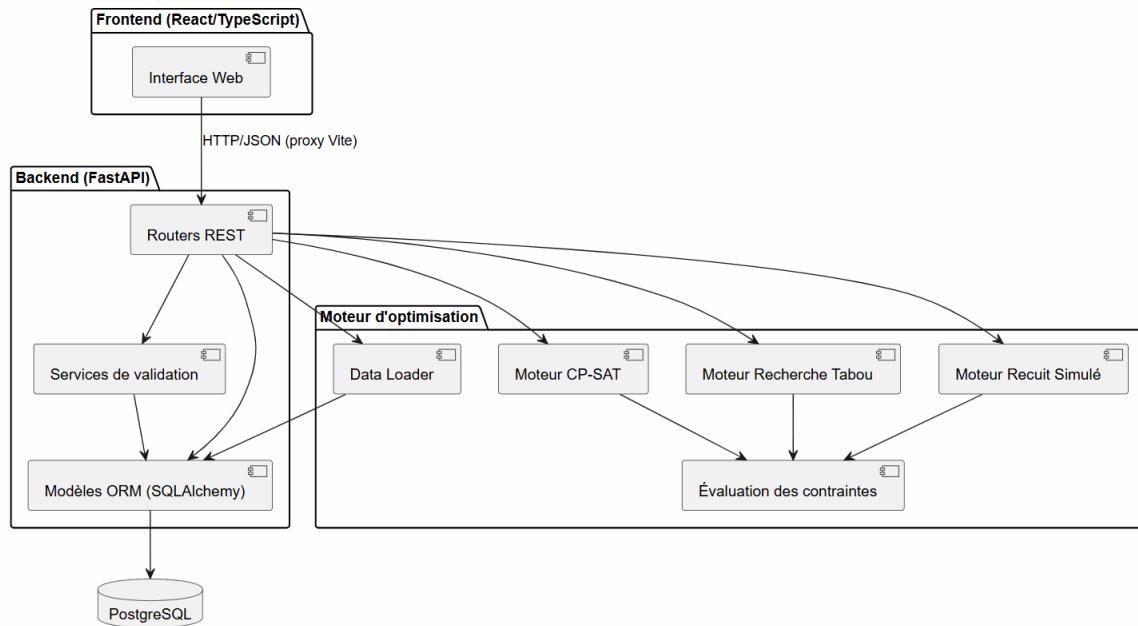


Figure 5 : Diagramme de composants

F. Diagramme de déploiement

En environnement de développement, l'ensemble des composants (serveur d'API, serveur de développement du client web, système de gestion de base de données) s'exécute sur un unique poste de travail, communiquant par le réseau local via le protocole HTTP. Le client web dialogue avec le serveur d'API sur le port 8000, tandis que le serveur de développement du client web sert l'interface sur le port 5173 et relaie les appels API par un mécanisme de proxy, évitant ainsi toute problématique de partage de ressources entre origines différentes (CORS) durant le développement.

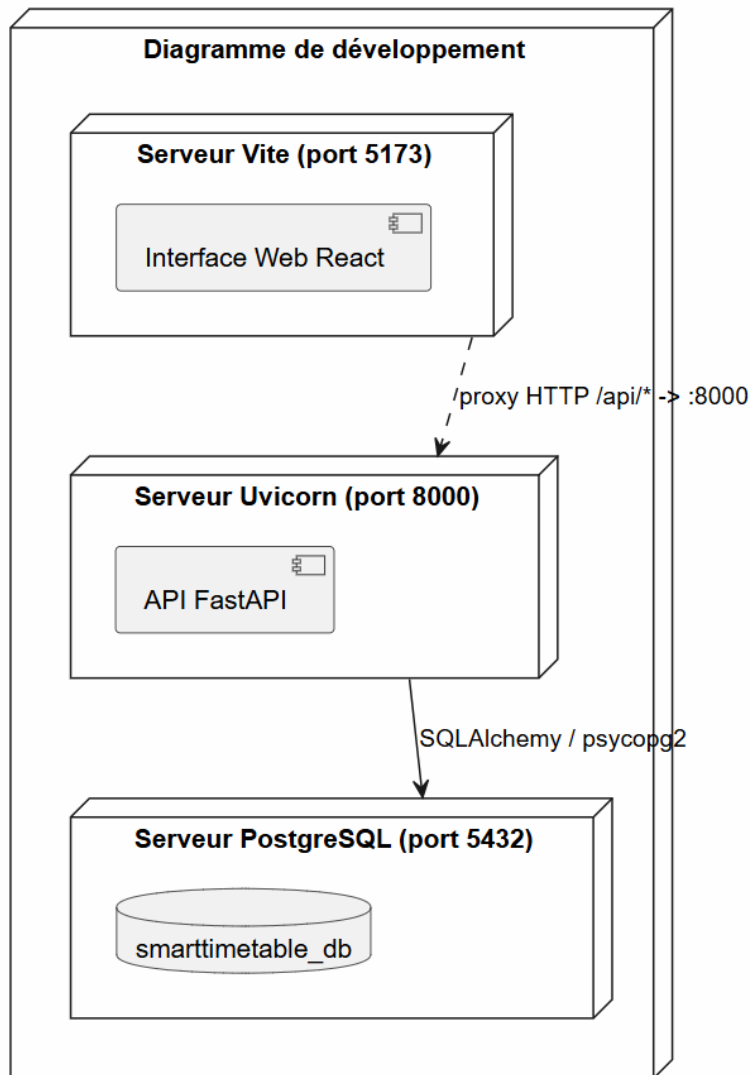


Figure 6 : Diagramme de déploiement

G. Modèle de base de données

Le modèle relationnel traduit directement le diagramme de classes. Les contraintes dures relatives à la non-superposition d'un enseignant, d'une salle ou d'un groupe sur un même créneau sont implémentées comme des contraintes d'unicité composites au niveau de la table des séances, garantissant leur respect indépendamment de la couche applicative.

1. Contraintes d'unicité

Contraintes d'unicité implémentées au niveau de la base de données :

Tableau 4: Contraintes d'unicité

Contrainte	Table Concernée	colonnes
Unicité enseignant * Créneau	Séances	Enseignant_id, Créneau_id
Unicité salle * Créneau	Séances	Salle_id,creneau_id
Unicité groupe * Créneau	Séances	Groupe_id,creneau_id
Unicité indisponibilité	Indisponibilités	Enseignant_id,creneau_id

2. Les contraintes dures (obligatoires)

Le référentiel complet des contraintes dures (obligatoires) prises en charge par le système est présenté dans le tableau suivant, Une solution violant une seule contrainte dure est rejetée. Le solveur doit obligatoirement toutes les satisfaire :

Tableau 5 : Les contraintes dures (obligatoires)

Code	Contrainte	Description
CD-01	Non-conflit Enseignant	Un prof = une seule séance par créneau
CD-02	Non-conflit Salle	Une salle = une seule séance par créneau
CD-03	Non-conflit Groupe	Un groupe = une seule séance par créneau
CD-04	Capacité de salle	Effectif groupe <= capacité salle
CD-05	Type de salle adapté	TP en salle TP, CM en amphi/salle
CD-06	Volume horaire	Nb séances = volume horaire du module
CD-07	Séances fixes	Créneaux imposés conservés

3. Contraintes souples :

Le référentiel des contraintes souples, exploitées par le moteur d'optimisation est présenté ci-dessous, Ces contraintes améliorent la qualité de l'emploi du temps. Chaque violation ajoute une pénalité à minimiser :

Tableau 6 : Contraintes souples

Code	Contrainte	Description
CS-01	Indisponibilités	Éviter les créneaux indisponibles
CS-02	Équilibrage journalier	Répartir les heures sur la semaine
CS-03	Regroupement	Regrouper les séances d'un module
CS-04	Minimiser les trous	Réduire les créneaux vides
CS-05	Préférences horaires	Respecter matin/après-midi
CS-06	Créneaux tardifs	Limiter les cours en fin de journée
CS-07	Pause déjeuner	Préserver 12h-14h libre

H. Conclusion

La conception présentée dans ce chapitre pose les fondations d'un système modulaire, dans lequel la logique de résolution combinatoire est totalement découplée de l'infrastructure web. Cette séparation, ainsi que la formalisation rigoureuse des contraintes dures et souples issues du cahier des charges, constituent les prérequis indispensables à la comparaison objective des algorithmes présentée au chapitre suivant.

III. Chapitre 3 – Réalisation

A. Environnement de développement

Le développement a été réalisé sous environnement Windows, avec Python 3.12 pour le backend et Node.js pour le frontend. L'éditeur de code utilisé est Visual Studio Code, avec les extensions Python et ESLint/Prettier pour le frontend. La gestion de versions s'appuie sur Git. Les dépendances Python sont isolées dans un environnement virtuel dédié (venv), et les dépendances JavaScript sont gérées par npm. La base de données PostgreSQL est exécutée localement durant le développement. La documentation interactive de l'API, générée automatiquement par FastAPI (interface Swagger), a été utilisée tout au long du projet pour tester chaque endpoint indépendamment de l'interface web.

B. Architecture logicielle

L'arborescence du backend reflète directement la séparation en couches :

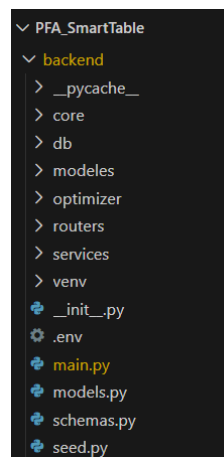


Figure 7: L'arborescence du backend

Tableau 7: Responsabilités des fichiers du backend

Dossier / fichier	Responsabilité
core/	Configuration de l'application (variables d'environnement, paramètres globaux).
modeles/models/ (modèles SQLAlchemy)	Déclaration des entités ORM et de leurs relations.
routers/	Endpoints REST, un fichier par entité métier.

Dossier / fichier	Responsabilité
services/	Logique de validation métier non exprimable comme contrainte SQL (adéquation capacité/type de salle).
optimizer/	Moteur d'optimisation, indépendant de FastAPI et de SQLAlchemy.
optimizer/engines/	Implémentation de chacun des trois algorithmes de résolution.
seed.py	Script de peuplement d'un jeu de données de test cohérent et faisable.

Le frontend suit une organisation similaire, avec un dossier api/ regroupant le client HTTP générique, un dossier components/ regroupant les composants réutilisables (barre de navigation, système de notifications, grille d'emploi du temps), et un dossier pages/ regroupant les trois grands écrans fonctionnels de l'application : Configuration & Gestion, Chargement de données et Traitement & Génération

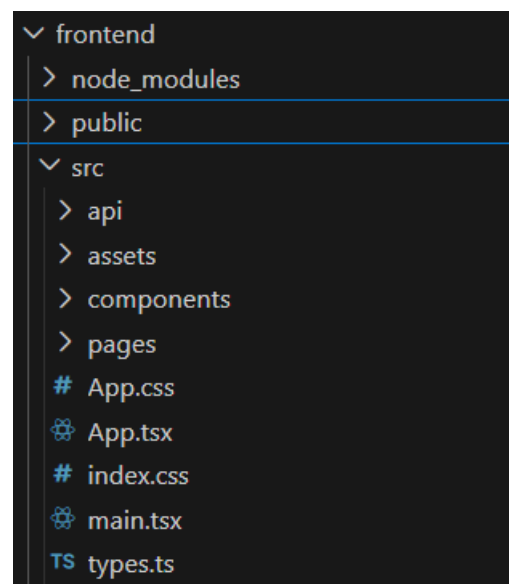


Figure 8 : L'arborescence du frontend

C. Implémentation des fonctionnalités

1. Gestion des entités de référence

Chaque entité de référence (départements, enseignants, salles, filières, groupes, modules) dispose d'un ensemble complet d'opérations de création, consultation, modification et suppression, exposées comme endpoints REST et consommées par des formulaires dédiés côté interface.

La relation de mutualisation entre modules et filières, de type plusieurs-à-plusieurs, est gérée par des endpoints spécifiques permettant de rattacher ou de détacher un module d'une filière, indépendamment de sa création.

The screenshot shows the 'Configuration & Gestion' tab in the SmartTimetable AI interface. The header bar includes the logo 'SmartTimetable AI' and three navigation buttons: 'Configuration & Gestion' (highlighted), 'Chargement de données', and 'Traitement & Génération'. Below the header, the main section is titled 'Configuration & Gestion' with a breadcrumb trail: 'Ordre recommandé : Départements → Enseignants / Salles → Filières → Groupes → Modules'. The '1. Départements' section contains a form with two input fields: 'Nom (ex: Informatique)' and 'Code (ex: INFC)', followed by an 'Ajouter' button. Below the form, a table lists existing departments:

Nom	Code	Action
INFO — Informatique & IA		Supprimer
Maths — Mathématiques		Supprimer

Figure 9 : Onglet Configuration et Gestion

The screenshot shows the '2. Enseignants' section of the 'Configuration & Gestion' tab. It features a form with five input fields: 'Nom', 'Prénom', 'Email', 'Grade (optionnel)', and 'Département...' (a dropdown menu), followed by an 'Ajouter' button. Below the form, a table lists existing teachers:

Nom	Prénom	Email	Grade (optionnel)	Département...	Action
Karim	Bennani	karim.bennani@ecole.ma			Supprimer
Salma	El Idrissi	salma.elidrissi@ecole.ma			Supprimer
Youssef	Chraïbi	youssef.chraïbi@ecole.ma			Supprimer
Nadia	Amrani	nadia.amrani@ecole.ma			Supprimer
Omar	Fassi	omar.fassi@ecole.ma			Supprimer
Imane	Tazi	imane.tazi@ecole.ma			Supprimer
Rachid	Benjelloun	rachid.benjelloun@ecole.ma			Supprimer
Hind	Lahlou	hind.lahlou@ecole.ma			Supprimer

2. Gestion des créneaux et des indisponibilités

Les créneaux horaires constituent une donnée de référence quasi statique, définie une fois par établissement (jour, heure de début, heure de fin, ordre dans la journée).

Les enseignants peuvent déclarer une indisponibilité sur un créneau donné, exploitée par le moteur d'optimisation comme contrainte souple (CS-01).

3. Gestion des séances et validation des contraintes dures

La création d'une séance, qu'elle soit manuelle ou issue d'une génération automatique, traverse trois niveaux de validation successifs.

Le premier niveau vérifie l'existence de chacune des références (module, enseignant, salle, groupe, créneau) et retourne une erreur explicite en cas de référence invalide.

Le deuxième niveau, implémenté dans la couche de services, contrôle les contraintes CD-04 (capacité de salle) et CD-05 (type de salle adapté), non exprimables comme de simples contraintes SQL car elles nécessitent la comparaison de deux entités distinctes.

Le troisième niveau s'appuie sur les contraintes d'unicité composites définies au niveau de la base de données (CD-01, CD-02, CD-03) : toute violation est interceptée et traduite en message d'erreur explicite, indiquant précisément quelle contrainte a été violée.

4. Import de données en masse

Afin de réduire le temps de saisie manuelle des données de référence, un module d'import a été développé, permettant de charger un fichier Excel (.xlsx) par type d'entité. Le fichier est analysé ligne par ligne ; chaque ligne est traitée dans sa propre transaction, de sorte qu'une erreur sur une ligne n'empêche pas le traitement des lignes suivantes. Un rapport détaillé est retourné à l'utilisateur, indiquant le nombre de lignes importées avec succès et, pour chaque échec, le numéro de ligne concerné et le message d'erreur précis.

The screenshot shows the 'SmartTimetable AI' application interface. The top navigation bar includes 'Configuration & Gestion', 'Chargement de données' (highlighted), and 'Traitement & Génération'. The main content area is titled '1. Choisir le type de données'. It features a row of tabs: 'Départements', 'Enseignants' (active), 'Salles', 'Filières', 'Groupes', and 'Modules'. Below the tabs, a text input field contains 'Colonnes attendues : nom, prenom, email, grade, departement_code'. The second section, '2. Déposer le fichier', contains a large dashed rectangular area for file upload. Inside this area, the text reads 'Glisse un fichier Excel ici, ou clique pour parcourir' and 'Formats acceptés : .xlsx, .xls'. An 'Importer' button is positioned at the bottom left of this section.

Figure 10 : Onglet Chargement de données

5. Génération des séances

Une fois un emploi du temps généré, l'utilisateur peut en visualiser le contenu sous forme de grille hebdomadaire, filtrée soit par filière (l'ensemble des groupes

de la filière étant affichés), soit par enseignant (l'ensemble de ses séances, tous groupes confondus).

Cette grille reprend l'identité visuelle de l'établissement et peut être imprimée ou exportée au format PDF directement depuis le navigateur.

SmartTimetable AI

Configuration & Gestion

Chargement de données

Traitement & Génération

Traitement & Génération

Génère, compare et publie l'emploi du temps à partir des données saisies.

Trois approches, une même grille à remplir

Chaque algorithme reçoit le même jeu de contraintes et produit une solution comparable sur trois critères : le temps de calcul, le respect des contraintes dures, et le score de pénalité sur les préférences.

Méthode exacte

CP-SAT

Résout le problème comme un système de contraintes strictes : explore l'espace des solutions valides et garantit soit un optimum, soit la preuve qu'aucune solution ne satisfait toutes les contraintes dures.

Le plus lent des trois, le plus fiable.

Métaheuristique

Recherche Tabou

Part d'une solution approximative et l'améliore par petits échanges de séances, en évitant de revenir sur ses derniers mouvements grâce à une mémoire à court terme.

Rapide, bonne qualité, sans garantie d'optimalité.

Métaheuristique

Recuit Simulé

S'inspire du refroidissement des métaux : accepte temporairement des solutions moins bonnes pour échapper aux optimums locaux, puis devient de plus en plus strict à mesure que la température baisse.

Simple à régler, efficace sur de grands volumes.

Lancer une génération

CP-SAT

Lancer la génération

Comparer les 3 algorithmes

CP-SAT

Recherche Tabou

Recuit Simulé

Emplois du temps

Par filière

Choisir...

Afficher

Lancer une génération

CP-SAT

Lancer la génération

Comparer les 3 algorithmes

Emplois du temps

Par filière

Choisir...

Afficher

Par filière

Par enseignant

Figure 11: Génère, compare et publie l'emploi du temps à partir des données saisies.

33

D. Implémentation des fonctionnalités d'intelligence artificielle

Le cœur scientifique de ce projet réside dans le moteur d'optimisation, qui implémente et compare trois algorithmes distincts, chacun respectant une interface commune (une méthode `generate()` acceptant les données du problème et retournant une solution accompagnée de ses indicateurs de performance). Cette interface commune permet de substituer un algorithme à un autre sans modification du reste du système, et constitue le fondement méthodologique de l'étude comparative.

1. Programmation par contraintes, CP-SAT (Google OR-Tools)

CP-SAT est un solveur de programmation par contraintes fondé sur les techniques de satisfaction booléenne (SAT), capable de garantir soit l'optimum, soit la preuve qu'aucune solution ne satisfait l'ensemble des contraintes dures. [3]

Dans le modèle implémenté, chaque séance à planifier se voit associer deux variables de décision : le créneau qui lui sera affecté et la salle qui l'accueillera, cette dernière étant restreinte dès la construction du modèle aux salles satisfaisant les contraintes CD-04 et CD-05. Les contraintes CD-01 et CD-03 (non-conflit enseignant et groupe) sont exprimées par des contraintes de différence globale (`AllDifferent`) sur les variables de créneau, regroupées respectivement par enseignant et par groupe. La contrainte CD-02 (non-conflit salle), dépendant simultanément des deux variables de décision, est exprimée par une contrainte de non-recouvrement pour chaque paire de séances.

La fonction objective minimise le nombre d'affectations correspondant à une indisponibilité déclarée (CS-01).

The screenshot displays the CP-SAT solver interface. At the top, there is a section titled "Lancer une génération" (Launch a generation) containing a dropdown menu set to "CP-SAT", an orange button labeled "Lancer la génération" (Launch the generation), and a dark blue button labeled "Comparer les 3 algorithmes" (Compare the 3 algorithms). Below this is a "Résultat" (Result) section with a table of performance metrics:

Algorithme	Temps de calcul	Contraintes dures	Score pénalité
cp_sat	0.52 s	Respectées	26.8

A green notification box at the bottom right states: "Activer Windows
Planning généré avec cp_sat — 28
séances. Cliquez sur les paramètres pour activer Windows."

Emploi du temps — Génie Logiciel				
Année Universitaire 2025-2026				
	08:30–10:15	10:30–12:15	14:30–16:15	16:30–18:15
Lundi	Génie Logiciel Avancé CM Youssef Chraïbi COURS-103	Algorithmique Avancée TP Karim Bennani TP-201	Anglais Technique TD Rachid Benjelloun COURS-103	Bases de Données TD Salma El Idrissi COURS-102
Mardi	Génie Logiciel Avancé TD Youssef Chraïbi COURS-103	Soft Skills & Gestion de Projet TD Hind Lahlou COURS-101	Algorithmique Avancée CM Karim Bennani COURS-102	Bases de Données TP Salma El Idrissi TP-201
Mercredi	Communication & Langues TD Mehdi Ouazzani COURS-103	Algorithmique Avancée TD Karim Bennani COURS-103	Bases de Données CM Salma El Idrissi COURS-101	

Figure 12 : solution générée par l'Algorithme CP-SAT

2. Recherche tabou

La recherche tabou est une métaheuristique de recherche locale qui part d'une solution initiale, ici générée aléatoirement dans le respect des contraintes de capacité et de type de salle, puis explore itérativement son voisinage en appliquant de petites modifications (changement du créneau ou de la salle d'une séance).

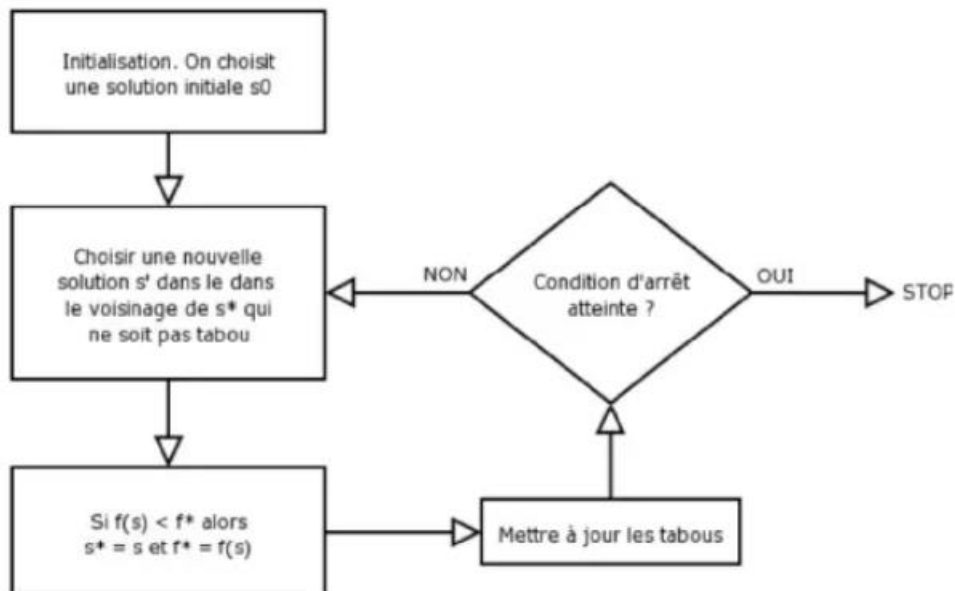


Figure 13 : Algorithme du Recherche tabou

Afin d'éviter de retomber dans un cycle de solutions déjà visitées, chaque mouvement effectué est conservé dans une liste taboue de taille limitée, interdisant temporairement sa répétition, sauf lorsqu'un critère d'aspiration est satisfait (le mouvement conduit à la meilleure solution trouvée jusqu'alors). Le coût d'une solution combine le score de pénalité des contraintes souples et une pénalité forte pour chaque violation de contrainte dure, permettant à la recherche de traverser temporairement des états invalides afin d'explorer plus largement l'espace des solutions.

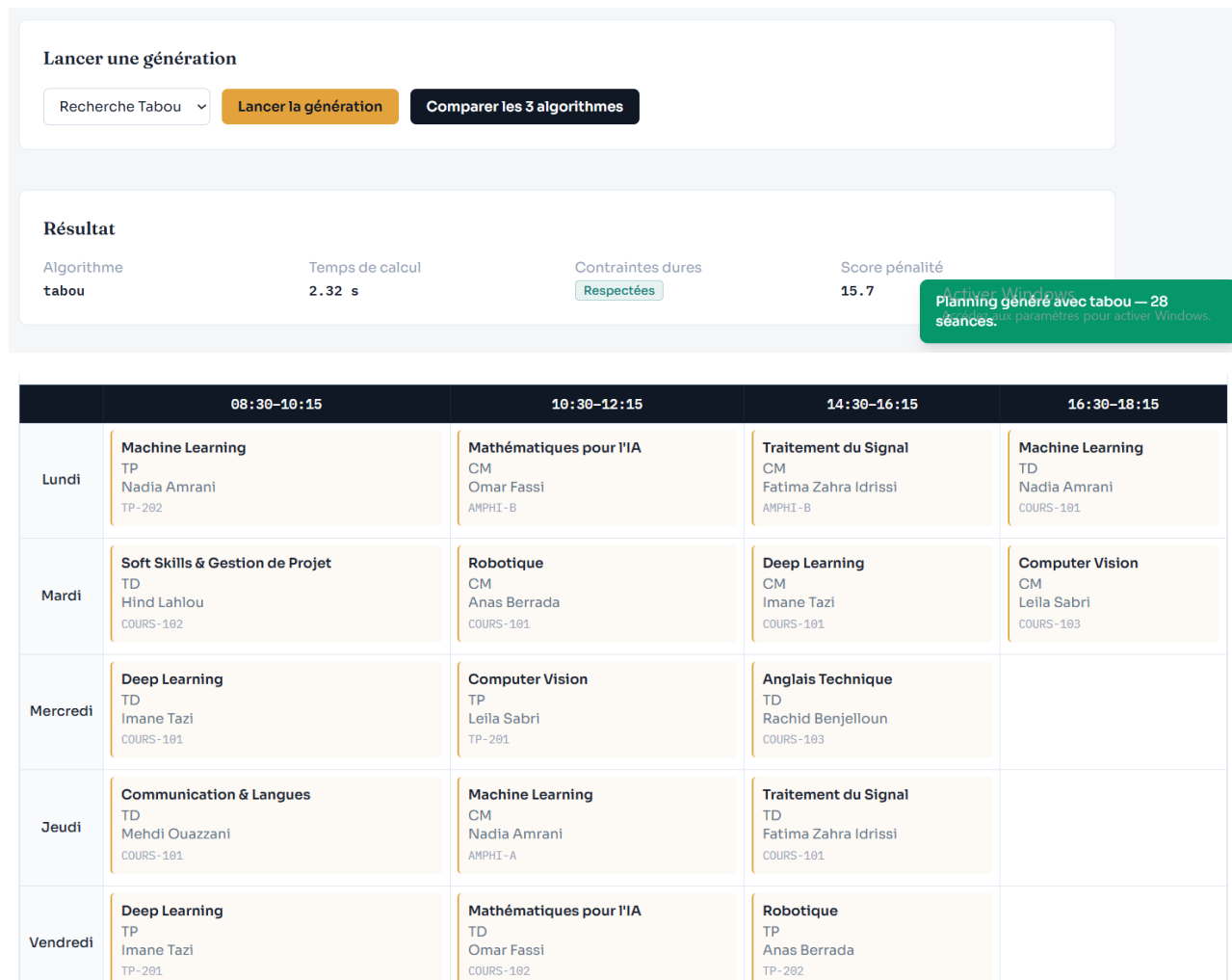


Figure 14 : solution générée par l'Algorithme Tabou

3. Recuit simulé

Le recuit simulé s'inspire du processus physique de refroidissement contrôlé des métaux. À l'instar de la recherche tabou, l'algorithme part d'une solution initiale et explore son voisinage par petites modifications successives. [6]

La différence fondamentale réside dans le critère d'acceptation d'un mouvement, une solution dégradant le coût courant peut néanmoins être acceptée avec une probabilité dépendant d'un paramètre de température, décroissant progressivement au fil des itérations selon un taux de refroidissement fixé. En début d'exécution, la température élevée favorise une exploration large de l'espace des solutions ; en fin d'exécution, la température proche de zéro restreint l'algorithme aux seuls mouvements améliorants, favorisant la convergence.

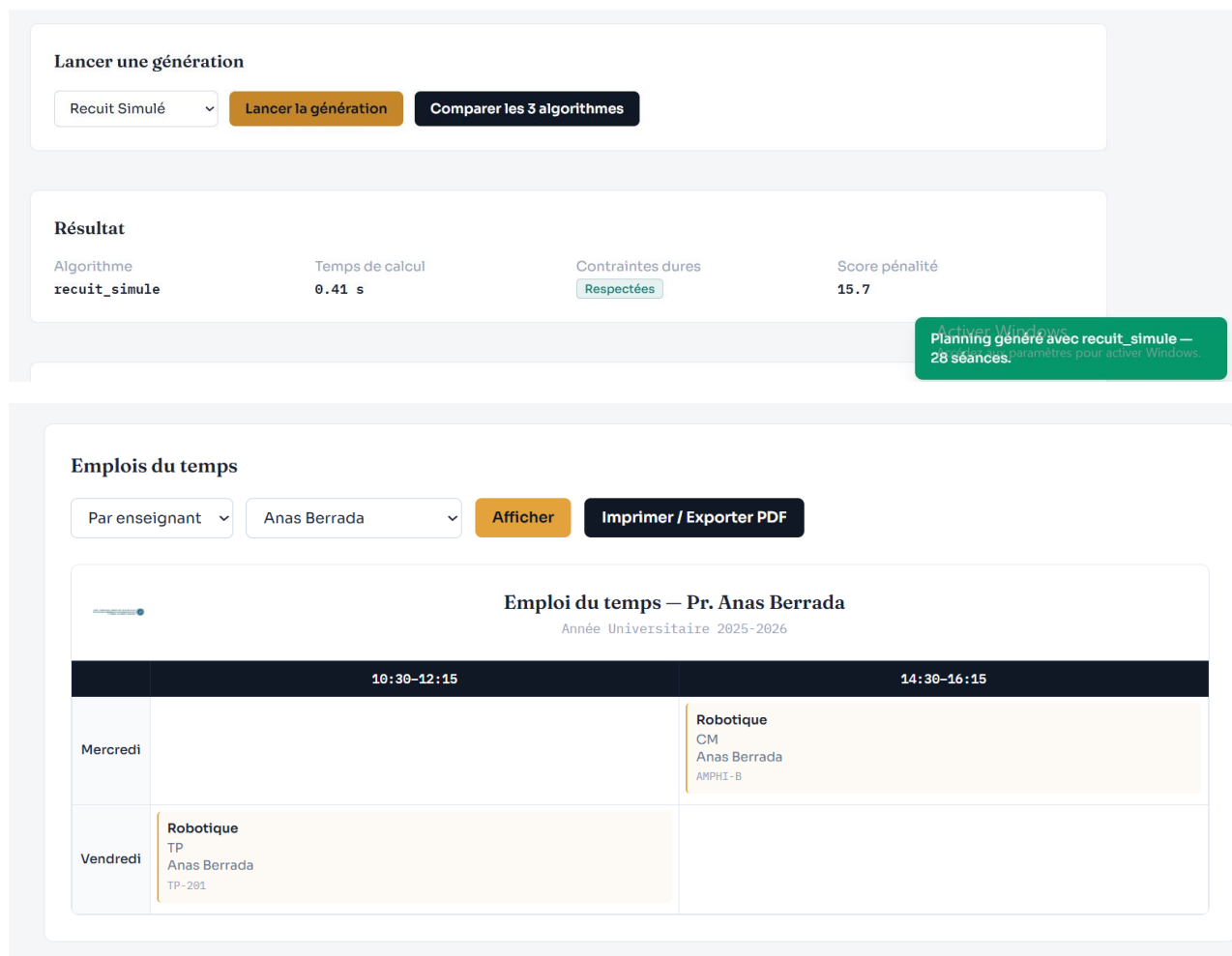


Figure 15 : solution générée par l'Algorithme recuit simulé, emplois pour Enseignant Anas Berrada

4. Fonction d'évaluation partagée

Afin de garantir une comparaison rigoureuse entre les trois algorithmes, une unique fonction d'évaluation est partagée par l'ensemble du moteur d'optimisation.

Elle se décompose en deux parties : une fonction de vérification stricte des contraintes dures, retournant un booléen accompagné de la liste des violations éventuelles, et une fonction de calcul du score de pénalité des contraintes souples, pondérant chaque violation selon un poids associé à sa contrainte. Cette mutualisation garantit qu'aucun algorithme n'est avantagé par une fonction de coût qui lui serait propre.

5. Étude comparative

Les trois algorithmes sont exécutés sur un jeu de données identique et comparés selon trois axes : le **temps de calcul**, le **respect strict des contraintes dures**, et le **score de pénalité** associé aux contraintes souples. Cette comparaison est directement accessible depuis l'interface, dans l'onglet Traitement & Génération, où un tableau récapitulatif présente les résultats des trois algorithmes côte à côte.

Planning généré
28 séances — vue brute, la grille visuelle est juste en dessous.

Type	Module	Groupe	Salle	Créneau
CM	#101	#24	#64	#243
TD	#101	#24	#64	#237
TP	#101	#24	#67	#233
CM	#102	#24	#62	#240
TD	#102	#24	#65	#245
TP	#102	#24	#68	#241
CM	#103	#24	#64	#235
TD	#103	#24	#66	#236

Figure 16 : Planning généré

Lancer une génération

Recuit Simulé ▼ **Lancer la génération** **Comparer les 3 algorithmes**

Tableau comparatif

Algorithme	Temps (s)	Score pénalité	Contraintes dures	Séances
cp_sat	0.39	26.8	Respectées	28
tabou	1.61	15.7	Respectées	28
recuit_simule	0.27	16	Respectées	28

Figure 17 : tableau comparatif des trois algorithmes

6. Précision sur le périmètre des fonctionnalités d'intelligence artificielle

Dans le périmètre effectivement réalisé pour ce Projet de Fin d'Année, l'effort a été concentré sur le cœur du problème d'optimisation combinatoire, jugé le plus directement pertinent au regard de la problématique du projet, au travers des trois algorithmes présentés ci-dessus.

7. Sécurité

Dans l'état actuel du développement, l'accent a été mis sur la robustesse fonctionnelle du système (validation des données, gestion explicite des erreurs, intégrité référentielle) plutôt que sur la sécurisation des accès. L'authentification des utilisateurs, la gestion différenciée des rôles (Administrateur, Enseignant, Étudiant) au niveau de l'API, ainsi que la protection des endpoints sensibles par un mécanisme de jetons d'accès, n'ont pas encore été implémentées à ce stade et constituent une perspective d'évolution prioritaire.

E. Conclusion

Ce chapitre a détaillé la réalisation technique du projet, depuis la gestion des données de référence jusqu'au moteur d'optimisation comparant trois approches algorithmiques distinctes. L'architecture modulaire retenue a permis de développer, tester et valider chaque composant de manière indépendante, avant son intégration progressive à l'ensemble du système.

Le chapitre suivant présente la stratégie de test adoptée ainsi que les résultats obtenus.

IV. Chapitre 4 : Tests et évaluation

A. Stratégie de test

La stratégie de test adoptée a suivi une démarche progressive et incrémentale, cohérente avec la méthodologie de développement décrite en introduction. Chaque brique fonctionnelle a été testée isolément avant son intégration : les endpoints REST ont été validés unitairement via l'interface Swagger générée automatiquement par FastAPI [2], les contraintes dures ont été testées par la tentative volontaire de créer des séances en conflit (vérification du code d'erreur HTTP 409 retourné), et le moteur d'optimisation a été testé en dehors de tout contexte web, par exécution directe d'un script Python autonome, avant son intégration à l'API

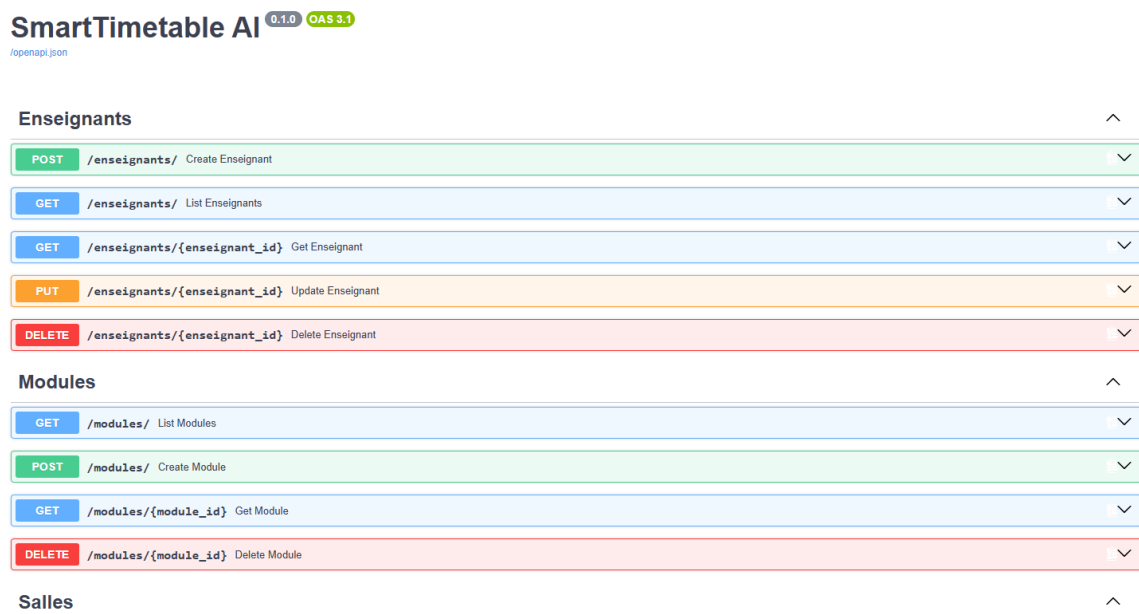


Figure 18 : l'interface Swagger générée automatiquement par FastAPI

B. Jeux de tests

Un script de peuplement automatisé (seed.py) a été développé afin de générer un jeu de données de test à la fois réaliste et volontairement calibré pour garantir l'existence d'au moins une solution valide. Ce script crée un département, une douzaine d'enseignants, sept salles de types variés (amphithéâtre, salle de cours, salle de travaux pratiques), deux filières (dont l'une avec des modules mutualisés en tronc commun), deux groupes et une douzaine de modules, ainsi que l'ensemble des créneaux hebdomadaires disponibles.

La calibration du jeu de données a nécessité un calcul explicite de faisabilité : le nombre total de séances requises par un groupe, déduit du volume horaire cumulé de ses modules, a été comparé au nombre de créneaux disponibles dans la semaine, afin de garantir qu'une solution respectant la contrainte de non-conflit de groupe (CD-03) existe bien mathématiquement.

C. Résultats

Le tableau ci-dessous présente le format des résultats obtenus lors de l'exécution de la comparaison des trois algorithmes sur le jeu de données de test.

Lancer une génération

CP-SAT

▼

Lancer la génération

Comparer les 3 algorithmes

Algorithme	Temps (s)	Score pénalité	Contraintes dures	Séances
cp_sat	0.51	43.8	Respectées	28
tabou	2.35	15.7	Respectées	28
recuit_simule	0.42	15.7	Respectées	28

Figure 19 : la comparaison des trois algorithmes sur le jeu de données de test

À titre d'analyse attendue, la méthode exacte (CP-SAT) devrait présenter le temps de calcul un peu plus élevé mais garantir systématiquement le respect des contraintes dures lorsqu'une solution existe. Les métaheuristiques devraient, à l'inverse, produire une solution en une temps sensiblement plus courte, au prix d'une garantie moindre d'optimalité sur les contraintes souples.

La comparaison entre recherche tabou et recuit simuler permet en outre d'évaluer l'apport respectif de la mémoire à court terme et de l'acceptation probabiliste de solutions dégradées sur ce problème spécifique.

D. Difficultés rencontrées

Le développement de ce projet a permis de rencontrer, diagnostiquer et résoudre un ensemble de difficultés techniques représentatives des défis rencontrés lors du passage d'une architecture applicative simple à un système multi-couches intégrant un moteur d'optimisation. Les principales difficultés rencontrées sont synthétisées ci-dessous :

1. Dérive de schéma sans outil de migration

En l'absence d'un outil de migration de schéma (tel qu'Alembic) durant une large partie du développement, plusieurs évolutions du modèle de données (renommage de colonnes, ajout de clés étrangères) ont provoqué des désynchronisations entre le schéma effectivement présent en base de données et les modèles SQLAlchemy déclarés dans le code, se traduisant par des erreurs de colonne inexistante lors de l'exécution. La résolution de ces incidents a permis de mettre en évidence l'importance d'un outil de migration dès qu'un schéma relationnel dépasse un stade exploratoire, recommandation intégrée aux perspectives d'amélioration.

2. Verrouillage de transaction en base de données

Un incident particulièrement instructif a concerné un blocage indéfini d'une requête d'insertion, sans qu'aucune erreur ne soit levée côté serveur applicatif. Le diagnostic a permis d'identifier qu'une session ouverte dans un client d'administration de base de données (transaction non validée) maintenait un verrou sur la table concernée, bloquant silencieusement toute écriture concurrente, PostgreSQL ne définissant aucun délai d'expiration par défaut sur ce type de verrou. Cet incident a conduit à l'ajout d'un délai d'expiration explicite (`statement_timeout`) au niveau de la connexion applicative, afin qu'un tel blocage se traduise désormais par une erreur explicite plutôt que par un silence indéfini.

3. Infaisabilité mathématique d'un jeu de données de test

Une tentative de génération a échoué de manière répétée avec un message « aucune solution trouvée », y compris avec la méthode exacte CP-SAT pourtant réputée garantir une réponse correcte. L'analyse a permis d'établir qu'il ne s'agissait pas d'un défaut du moteur de résolution, mais d'une véritable impossibilité combinatoire : le volume horaire cumulé des modules d'un groupe dépassait le nombre de créneaux disponibles dans la semaine, rendant toute solution respectant la contrainte de non-conflit de groupe (CD-03) mathématiquement impossible, indépendamment de l'algorithme employé. Cette situation, assimilable à un problème de dénombrement de type « principe des tiroirs », a nécessité un recalcul explicite de la capacité du jeu de données avant d'ajuster soit le nombre de créneaux disponibles, soit le volume horaire des modules concernés.

E. Perspectives d'amélioration

→ Mettre en place un outil de migration de schéma (Alembic), afin de fiabiliser l'évolution du modèle de données sans recreation manuelle des tables ;

→ implémenter un système d'authentification et de gestion différenciée des rôles (Administrateur, Enseignant, Étudiant), avec protection des endpoints sensibles de l'API ;

- Développer le module « Annonces », permettant de signaler une indisponibilité d'enseignant ou l'annulation d'un cours directement depuis l'interface, avec notification des groupes concernés ;
- Intégrer un agent conversationnel capable de répondre aux questions relatives aux cours et à l'emploi du temps, exploitant les données du système par une architecture de génération augmentée par récupération (RAG) ;
- Étendre la fonction objective du solveur CP-SAT afin d'intégrer directement l'ensemble des contraintes souples (CS-02 à CS-07), actuellement évaluées a posteriori plutôt qu'optimisées nativement par la méthode exacte ;
- Exploiter la table de paramétrage des contraintes (Contrainte) pour permettre à un responsable pédagogique d'ajuster dynamiquement les poids des contraintes souples sans modification du code source ;
- Conteneuriser l'application (Docker) afin de faciliter son déploiement sur un environnement de production ;
- Étendre les tests automatisés (tests unitaires et d'intégration) afin de sécuriser les évolutions futures du système ;

V. Conclusion générale

A. Bilan

Ce Projet de Fin d'Année a permis de concevoir et de développer SmartTimetable AI, une plateforme complète de génération et d'optimisation automatique des emplois du temps académiques. L'ensemble des objectifs fixés en introduction a été atteint : modélisation relationnelle fidèle des entités académiques et de leurs contraintes, développement d'une API REST robuste appliquant ces contraintes à plusieurs niveaux (base de données, couche de services), conception d'un moteur d'optimisation découplé implémentant trois algorithmes distincts (CP-SAT, recherche tabou, recuit simulé) au sein d'une interface commune, et développement d'une interface web complète permettant la gestion des données, l'import en masse et la visualisation imprimable des emplois du temps générés.

B. Apports

Sur le plan technique, ce projet a permis d'approfondir des compétences en modélisation de données relationnelles complexes, en développement d'API REST typées, et surtout en modélisation et résolution de problèmes d'optimisation combinatoire par des approches à la fois exactes et heuristiques. Sur le plan méthodologique, l'approche par étapes progressives, validées indépendamment avant intégration, s'est révélée déterminante pour maîtriser la complexité croissante du système. Sur le plan scientifique enfin, la comparaison rigoureuse de trois algorithmes selon une fonction d'évaluation strictement partagée constitue une contribution directement réutilisable pour l'étude comparative attendue dans le cadre de ce PFA.

C. Limites

Le système présente, dans son état actuel, plusieurs limites assumées et documentées. L'absence de mécanisme d'authentification et de gestion des rôles restreint le système à un usage en environnement de confiance.

La fonction objectif [6] du solveur exact n'intègre pour l'instant que la contrainte souple CS-01 ; les autres contraintes souples sont évaluées a posteriori plutôt qu'optimisées nativement. Enfin, l'absence d'un outil de migration de schéma a constitué, comme détaillé au chapitre 4, une source récurrente de désynchronisation durant le développement, désormais identifiée comme priorité d'amélioration.

D. Perspectives

Les perspectives d'évolution de ce projet, portent notamment sur la sécurisation du système par authentification et gestion des rôles, l'enrichissement de la fonction objectif du solveur exact pour couvrir l'ensemble des contraintes souples, le développement d'un module de gestion des annonces et d'un agent conversationnel exploitant les données du système, ainsi que la conteneurisation de l'application en vue d'un déploiement en production. Ces perspectives ouvrent la voie à une plateforme complète, prête à être déployée au sein d'un établissement réel.

E. Bibliographie

- [1] «edrawMax,» [En ligne]. Available: <https://www.edrawsoft.com/fr/ad/edrawmax-brand.html>.
- [2] «FastAPI,» [En ligne]. Available: <https://fastapi.tiangolo.com/>.
- [3] «developers.google.com,» [En ligne]. Available: <https://developers.google.com/optimization>.
- [4] «PlantUML,» [En ligne]. Available: <https://plantuml.com/fr/>.
- [5] J. A. e. M. André, «Recherche-Tabou».
- [6] M. S. Salimo, «Sur la Route de l'Optimum : Recuit Simulé pour le TSP,» [En ligne]. Available: <https://medium.com/wanabilini/sur-la-route-de-loptimum-recuit-simul%C3%A9-pour-le-tsp-9cb037e74979>.
- [7] A. Gondran, «pb_contraintes_binaires.pdf».

Les références suivantes ont été mobilisées pour la conception scientifique et technique de ce projet.